

Assignment 2

Question 2

2 (a) To obtain the data set, first install the package (with `install.packages()`) and then load it (with `library()`). Display some of the data set which is called `lowbwt`.

We may as well have `tidyverse` and `MASS` ready as well

```
library(MASS)
library(tidyverse)
library(aplore3)
```

And let's view some rows of `lowbwt` with `glimpse` to know we have the right thing

```
glimpse(lowbwt)

## Observations: 189
## Variables: 11
## $ id      <int> 4, 10, 11, 13, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25, 2...
## $ low     <fct> < 2500 g, < 2500 g, < 2500 g, < 2500 g, < 2500 g, < 2500...
## $ age     <int> 28, 29, 34, 25, 25, 27, 23, 24, 24, 21, 32, 19, 25, 16, ...
## $ lwt     <int> 120, 130, 187, 105, 85, 150, 97, 128, 132, 165, 105, 91,...
## $ race    <fct> Other, White, Black, Other, Other, Other, Other, Black, ...
## $ smoke   <fct> Yes, No, Yes, No, No, No, No, No, No, Yes, Yes, Yes, No,...
## $ ptl     <fct> "One", "None", "None", "One", "None", "None", "None", "0...
## $ ht      <fct> No, No, Yes, Yes, No, No, No, No, Yes, Yes, No, No, No, ...
## $ ui      <fct> Yes, Yes, No, No, Yes, No, Yes, No, No, No, No, Yes, No,...
## $ ftv     <fct> "None", "Two, etc.", "None", "None", "None", "None", "On...
## $ bwt     <int> 709, 1021, 1135, 1330, 1474, 1588, 1588, 1701, 1729, 179...
```

seems like we have the right data set.

2 (b) Each line of the data file represents one woman. How do you know that a logistic regression will predict the probability of a low birth weight, rather than a non-low birth weight. Explain briefly.

As each row of the dataset represents a singular woman, the data is almost what is wanted for a logistic regression, but there are a couple things we have to confirm.

In terms of the data itself, and if we look at the type of variable `low` is, we see that `low` is of a type `fctr`. To conduct a logistic regression in R, it is required that the response variable must be a `fctr`. That is one requirement to predicting the probability of a low birth weight.

Another requirement is that logistic regression is suited for a binary decision, that is, there should only be two possible values for `low`; [less than 2500 g] or [greater than or equal to 2500 g]. To confirm that, we must use `levels`

```
levels(lowbwt$low)
```

```
## [1] ">= 2500 g" "< 2500 g"
```

As there are only two possible categorical values in `low`, we can conclude that the requirements for a logistic regression in R has been fulfilled, and that a logistic regression will predict the probability of a low birth weight.

Side note: It should also be noted that the first level is ≥ 2500 g, which is going to be our baseline for predictions, and that < 2500 g will be the value that is going to be predicted in our logistic regression.

2 (c) Use logistic regression to predict the probability of a low birth weight as it depends on the mother's age, weight at last menstrual period, race, and number of first trimester physician visits. Display the results.

(Notes:

- i. you will probably have to consult the help for this data set to find out which variables these are. To do this, make a new code chunk, and in it, put a question mark followed by the name of the data frame. Running this should display the help for the data in the Help pane bottom right
- ii. The number of physician visits is categorized as none, one, two or more. That's OK. Use it as is.)

So, we use the hint in the notes:

```
?lowbwt
```

Given the `man` page of `lowbwt`, we find the description of the following variables matching to the variables asked:

- `age` - Age of mother (Years)
- `lwt` - Weight of mother at last menstrual period (Pounds)
- `race` - Race of the mother
- `ftv` - Number of physician visits during the trimester.

So, we now conduct the logistic regression with the asked variables:

```
lowbwt.1=glm(low~age+lwt+race+ftv, family="binomial", data=lowbwt)
summary(lowbwt.1)
```

```
##
## Call:
## glm(formula = low ~ age + lwt + race + ftv, family = "binomial",
##      data = lowbwt)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.4618  -0.9024  -0.6822   1.2368   2.0473
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.357130   1.068211   1.270   0.2039
## age         -0.017949   0.034211  -0.525   0.5998
## lwt         -0.014755   0.006537  -2.257   0.0240 *
## raceBlack    1.005666   0.499753   2.012   0.0442 *
## raceOther    0.383938   0.365445   1.051   0.2934
## ftvOne       -0.510097   0.425011  -1.200   0.2301
## ftvTwo, etc. -0.152842   0.422320  -0.362   0.7174
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 234.67  on 188  degrees of freedom
## Residual deviance: 221.17  on 182  degrees of freedom
## AIC: 235.17
##
## Number of Fisher Scoring iterations: 4
```

2 (d) We are going to remove explanatory variables that do not add anything significant to the model. Explain briefly why we should use `drop1` rather than `summary` to assess this.

The focus of `drop1` is to determine which explanatory variables are significant by giving p-values of each variable. `drop1` accounts for categorical variables, unlike `summary`.

If we look at the results above using `summary`, we see the **categorical values** being deemed as **categorical variables** (e.g. `raceBlank` and `raceOther`). This is a big concern because we may come to a possible conclusion that one categorical value is significant and another isn't of the same variable; we would not know whether the variable itself is significant or not.

`drop1` allows us to reliably know whether a categorical fits with the model, and thus, it is best used for this logistic regression model.

2 (e) Use backward elimination to build a better model for low birth weight. Do this using `step`, or by hand using `drop1` and `update`, whichever you prefer. Display the output for your final model. Use $\alpha = 0.10$. You should find that `race` and `lwt` are the two explanatory variables that remain

Let's be courageous and use `step`

```
lowbwt.2 = step(lowbwt.1, direction="backward", test="Chisq")

## Start:  AIC=235.17
## low ~ age + lwt + race + ftv
##
##      Df Deviance    AIC    LRT Pr(>Chi)
## - ftv    2   222.66 232.66 1.4875   0.4753
## - age    1   221.45 233.45 0.2780   0.5980
## <none>      221.17 235.17
## - race    2   225.42 235.42 4.2440   0.1198
## - lwt     1   226.93 238.93 5.7596   0.0164 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Step:  AIC=232.66
## low ~ age + lwt + race
##
##      Df Deviance    AIC    LRT Pr(>Chi)
## - age    1   223.26 231.26 0.5985   0.43917
## <none>      222.66 232.66
## - race    2   227.12 233.12 4.4628   0.10738
## - lwt     1   228.13 236.13 5.4669   0.01938 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Step:   AIC=231.26
## low ~ lwt + race
##
##           Df Deviance    AIC    LRT Pr(>Chi)
## <none>      223.26 231.26
## - race    2    228.69 232.69 5.4316  0.06615 .
## - lwt     1    229.66 235.66 6.4025  0.01140 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Quite a bit of output, but as expected, `race` and `lwt` are the final two explanatory variables than remain as `race`'s and `lwt`'s p-value is less than $\alpha = 0.10$. Thus, we can conclude that `race` and `lwt` are strongly significant when determining if a baby has a low birth weight or not.

2 (f) From the output of your final model, describe the effects of each remaining explanatory variable on the probability of the baby being low birth weight.

Given that `summary` enables us to determine the slopes of each variable (or categorical value), we run it as follows:

```
summary(lowbwt.2)

##
## Call:
## glm(formula = low ~ lwt + race, family = "binomial", data = lowbwt)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.3491  -0.8919  -0.7196   1.2526   2.0993
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.805753   0.845167   0.953   0.3404
## lwt         -0.015223   0.006439  -2.364   0.0181 *
## raceBlack    1.081066   0.488052   2.215   0.0268 *
## raceOther    0.480603   0.356674   1.347   0.1778
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 234.67  on 188  degrees of freedom
## Residual deviance: 223.26  on 185  degrees of freedom
## AIC: 231.26
##
## Number of Fisher Scoring iterations: 4
```

I would like to recall something I pointed out earlier:

```
levels(lowbwt$low)

## [1] ">= 2500 g" "< 2500 g"
```

“It should also be noted that the first level is ≥ 2500 g, which is going to be our baseline for predictions, and that < 2500 g will be the value that is going to be predicted in our logistic regression.”

Now, I would now like to go back to the model. `lwt` has a negative slope (at -0.0152), which means that the higher that `lwt` is, the less likely the baby will have a weight less than 2,500 g.

As a sanity check, we look at the description of `lwt`: “Weight of mother at last menstrual period”. This does make sense; if the mother weighs more, there is a higher chance that the baby will weigh more as well since the baby’s nutrition depends on what the mother consumes.

As for `race`, both `raceBlack` and `raceOther` have positive “slopes” (or better yet, an addition to the intercept) at 1.0810 and 0.4806 respectively. That is, if the baby is non-white, there is a higher likelihood that the baby will have a weight less than 2,500 g. `raceWhite` is held as a baseline within the intercept, and therefore has the lowest probability among all races.

2 (g) Find the quartiles of `lwt`, and obtain a list of the races. Save these in variables for the next part

Let’s use a STAC32 way

Let’s get the quantile values first:

```
lowbwt %>% summarize(Q1 = quantile(lwt, 0.25),
                     median = quantile(lwt, 0.5),
                     Q3 = quantile(lwt, 0.75)) -> quantile_vals
quantile_vals = list(lwt=t(quantile_vals))
quantile_vals
```

```
## $lwt
##      [,1]
## Q1      110
## median  121
## Q3      140
```

Then let’s get a list of races. We can use `levels` with this, and as requested, we save it as a list

```
racess = levels(lowbwt$race)
racess = list(race=racess)
racess
```

```
## $race
## [1] "White" "Black" "Other"
```

2 (h) Make a data frame containing all combinations of weight at last menstrual period and race whose values you obtained in the previous part. Predict the probability of a low-birth-weight baby for each of these, and display the results side by side with the values they are

I do know a tiny bit of SQL/relational algebra, and what is asked is something like the cartesian product. This took a tiny bit of googling.

```
lowbwt.3 = merge(quantile_vals,racess,all=TRUE)
lowbwt.3
```

```
##   lwt  race
## 1 110 White
## 2 121 White
```

```
## 3 140 White
## 4 110 Black
## 5 121 Black
## 6 140 Black
## 7 110 Other
## 8 121 Other
## 9 140 Other
```

Now, we would like to make the predictions

```
p=predict(lowbwt.2, lowbwt.3, type="response")
p
```

```
##          1          2          3          4          5          6          7
## 0.2955066 0.2618758 0.2099079 0.5528712 0.5112042 0.4391987 0.4041544
##          8          9
## 0.3645572 0.3005085
```

And we would like to see the predictions side-by-side with our new dataframe

```
cbind(lowbwt.3, p)
```

```
##   lwt  race      p
## 1 110 White 0.2955066
## 2 121 White 0.2618758
## 3 140 White 0.2099079
## 4 110 Black 0.5528712
## 5 121 Black 0.5112042
## 6 140 Black 0.4391987
## 7 110 Other 0.4041544
## 8 121 Other 0.3645572
## 9 140 Other 0.3005085
```

2 (i) Explain briefly how the predicted probabilities for each combination of `lwt` and `race` are consistent with your assessment of the slopes in part (f)

Firstly, let's get `raceWhite` out of the way; `raceWhite` is "categorized" within the intercept (as it is within the baseline), and hence it has the lowest probability among all races. `raceBlack` and `raceOther` positively add onto the probability as shown in (f), and thus shows consistency.

Let's look at `raceBlack` and `raceOther`. In (f), the output showed that `raceOther` has a lower probability than `raceBlack`, and our output appears consistent if we compare probabilities of `raceBlack` for each `lwt` value with the probabilities of `raceOther` for each `lwt` value. It is clear that `raceOther` has a lower probability in comparison to `raceBlack`, and this is consistent with the assessment in (f).

It also appears that, regardless of race, the higher the `lwt` is, the probability of getting a low birth weight decreases, which is also consistent with the assessment made in (f).

Therefore, given our probabilities, our results appear consistent with (f).

Question 4

4 (a) Read in the data and verify that you have the right number of rows, and more columns than will fit on the page

We read the csv file as follows:

```
url = "http://www.utoronto.ca/~butler/d29/wine_quality.csv"
wine = read_csv(url)
```

```
## Parsed with column specification:
## cols(
##   fixed_acidity = col_double(),
##   volatile_acidity = col_double(),
##   citric_acid = col_double(),
##   residual_sugar = col_double(),
##   chlorides = col_double(),
##   free_sulfur_dioxide = col_double(),
##   total_sulfur_dioxide = col_double(),
##   density = col_double(),
##   pH = col_double(),
##   sulphates = col_double(),
##   alcohol = col_double(),
##   qual = col_character()
## )
```

wine

```
## # A tibble: 4,898 x 12
##   fixed_acidity volatile_acidity citric_acid residual_sugar chlorides
##   <dbl>          <dbl>          <dbl>          <dbl>          <dbl>
## 1         7         0.27         0.36         20.7         0.045
## 2        6.3         0.3         0.34          1.6         0.049
## 3        8.1         0.28         0.4           6.9         0.05
## 4        7.2         0.23         0.32          8.5         0.058
## 5        7.2         0.23         0.32          8.5         0.058
## 6        8.1         0.28         0.4           6.9         0.05
## 7        6.2         0.32         0.16           7         0.045
## 8         7         0.27         0.36         20.7         0.045
## 9        6.3         0.3         0.34          1.6         0.049
## 10       8.1         0.22         0.43          1.5         0.044
## # ... with 4,888 more rows, and 7 more variables:
## #   free_sulfur_dioxide <dbl>, total_sulfur_dioxide <dbl>, density <dbl>,
## #   pH <dbl>, sulphates <dbl>, alcohol <dbl>, qual <chr>
```

Since I am knitting this to a PDF, it shows there are an addition 4,888 rows (plus 10 that's currently on there). Thus, we have the same number of rows as described.

Likewise, we see 7 other variables that cannot fit the page that include:

- free_sulfur_dioxide
- total_sulfur_dioxide
- density
- pH
- sulphates
- alcohol

- qual

Thus, the dataset that we have seems identical to what was described, as wanted.

4 (b) Why would an ordinal logistic regression be better than a nominal one for these data? Explain briefly

This is because it depends on the response variable. In our case, we are trying to predict wine quality:

```
levels(factor(wine$qual))
```

```
## [1] "excellent" "good"      "moderate"  "poor"
```

and it can be ordered from **poor** to **excellent**; an ordinal logistic regression will be better than a nominal one in this case since the quality of wine can be naturally ordered. Nominal regression treats response variables as “equal weighting” where order does not have to make sense, thus, conducting a nominal regression model is not suitable.

4 (c) Create a suitable ordered response factor, or explain briefly why you can turn qual into a factor directly.

As we see from our output in (b), it’s already ordered properly from greatest to least: **excellent** > **good** > **moderate** > **poor**. This is because using a factor orders the levels alphabetically, and coincidentally, our factor shows greatest to least.

However, I am not comfortable with “greatest to least”. Instead, I want “least to greatest” so I will have a crack at it:

```
quals = wine %>% distinct(qual) %>% pull(qual)
quals
```

```
## [1] "good"      "moderate"  "excellent" "poor"
```

There are a few things we need to switch around

```
quals1 = quals[c(4, 2, 1, 3)]
quals1
```

```
## [1] "poor"      "moderate"  "good"      "excellent"
```

And let’s attach quals1 to our data frame

```
wine = wine %>% mutate(qual_ord=ordered(qual, quals1))
# let's use glimpse so that we can see my new variable as an ordered factor
glimpse(wine)
```

```
## Observations: 4,898
## Variables: 13
## $ fixed_acidity      <dbl> 7.0, 6.3, 8.1, 7.2, 7.2, 8.1, 6.2, 7.0, 6...
## $ volatile_acidity   <dbl> 0.27, 0.30, 0.28, 0.23, 0.23, 0.28, 0.32,...
## $ citric_acid        <dbl> 0.36, 0.34, 0.40, 0.32, 0.32, 0.40, 0.16,...
## $ residual_sugar     <dbl> 20.70, 1.60, 6.90, 8.50, 8.50, 6.90, 7.00...
## $ chlorides          <dbl> 0.045, 0.049, 0.050, 0.058, 0.058, 0.050,...
## $ free_sulfur_dioxide <dbl> 45, 14, 30, 47, 47, 30, 30, 45, 14, 28, 1...
## $ total_sulfur_dioxide <dbl> 170, 132, 97, 186, 186, 97, 136, 170, 132...
## $ density            <dbl> 1.0010, 0.9940, 0.9951, 0.9956, 0.9956, 0...
## $ pH                 <dbl> 3.00, 3.30, 3.26, 3.19, 3.19, 3.26, 3.18,...
```

```
## $ sulphates      <dbl> 0.45, 0.49, 0.44, 0.40, 0.40, 0.44, 0.47,...
## $ alcohol       <dbl> 8.8, 9.5, 10.1, 9.9, 9.9, 10.1, 9.6, 8.8,...
## $ qual          <chr> "good", "good", "good", "good", "good", "...
## $ qual_ord      <ord> good, good, good, good, good, good, good,...
```

4 (d) Fit an ordered logistic regression predicting quality from volatile acidity and residual sugar. To enable the grader to check that you have done the right thing, “print” the results: that is, type the name of your fitted model object on a line by itself and include the output from this in what you hand in.

Thankfully, the variable names are easy to interpret, so we use `polr`:

```
wine.1=polr(qual_ord~volatile_acidity + residual_sugar, data = wine)
```

As requested, we show our output:

```
wine.1

## Call:
## polr(formula = qual_ord ~ volatile_acidity + residual_sugar,
##       data = wine)
##
## Coefficients:
## volatile_acidity  residual_sugar
##      -4.09477866      -0.03534521
##
## Intercepts:
## poor|moderate  moderate|good  good|excellent
##    -4.71722510    -2.06206176    -0.01001568
##
## Residual Deviance: 11230.23
## AIC: 11240.23
```

4 (e) Can either one of these two explanatory variables be dropped from the model? Obtain some output that will help you decide

`drop1` seems like a suitable candidate for this

```
drop1(wine.1, test="Chisq")

## Single term deletions
##
## Model:
## qual_ord ~ volatile_acidity + residual_sugar
##           Df    AIC    LRT Pr(>Chi)
## <none>          11240
## volatile_acidity  1 11450 211.997 < 2.2e-16 ***
## residual_sugar    1 11283  44.716 2.278e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

As we can see, both variables are considered significant to determine the quality of wine because the two have a p-value less than $\alpha = 0.05$ (at 2.2e-16 and 2.278e-11 for `volatile_acidity` and `residual_sugar` respectively).

Since both of the explanatory variables are significant, these two explanatory variables cannot be dropped from the model.

4 (f) The quartiles of `volatile_acidity` are 0.21 and 0.32, and of `residual_sugar` are 1.7 and 9.9. Make a data frame containing all combinations of these values, and use it to predict that wines with those values of `volatile_acidity` and `residual_sugar` will be awarded each of the ratings.

Since I “cheated” a bit from the previous question by using `merge`; I have been informed that `crossing` is another way to say “cross join”:

```
vol_aci = c(0.21, 0.32)
res_sug = c(1.7, 9.9)
# we have to typecase combos because crossing only returns a tibble
combos = data.frame(crossing(volatile_acidity = vol_aci, residual_sugar = res_sug))
# and we display our combinations
combos
```

```
##   volatile_acidity residual_sugar
## 1              0.21             1.7
## 2              0.21             9.9
## 3              0.32             1.7
## 4              0.32             9.9
```

Now, as instructed by the question, I will be using our original model from (d) to predict our combinations' probabilities.

```
p=predict(wine.1, combos, type="probs")
cbind(combos, p)
```

```
##   volatile_acidity residual_sugar      poor moderate      good excellent
## 1              0.21             1.7 0.02194053 0.2199997 0.4710547 0.2870050
## 2              0.21             9.9 0.02910230 0.2698607 0.4695271 0.2315098
## 3              0.32             1.7 0.03399981 0.2996673 0.4621577 0.2041752
## 4              0.32             9.9 0.04491715 0.3559592 0.4380459 0.1610777
```

4 (g) Describe the effect of increasing residual sugar on the wine quality rating

To scope this down, let's focus on only the `volatile_acidity` being fixated at 0.21 (as it is “fair” to, we are not changing anything other than residual sugar).

We see that an increase in residual sugar will leave higher probabilities in categories such as “poor” and “moderate” (for example, look at the probability of “poor” increasing from 0.02194053 to 0.02910230 when you increase the `residual_sugar`). However, there are lower probabilities in categories such as “good” and “excellent”.

We can draw the same conclusions if we were to fixate `volatile_acidity` at 0.32.

Therefore, we can conclude that an increase in residual sugar will decrease the wine quality rating.

4 (h) Describe the effect of increasing volatile acidity on the wine quality rating

Let's take the same logic as we did in (g) and fixate `residual_sugar` being at 1.7.

We see that an increase in volatile acidity will have higher probabilities in categories such as “poor” and “moderate”, but lower probabilities such as “good” and “excellent”.

We can also draw the same conclusions if we fixate `residual_sugar` at 9.9.

Therefore, we can conclude that an increase in `volatile acidity` will decrease the wine quality rating.